Predicting Coronary Heart Disease from BRFSS Survey Data

Linus Baumberger, Martin Sanchez Lopez, Ghalia Bennani
Department of Computer Science, EPFL, Switzerland

***Abstract*—We predict whether a person has coronary heart disease (MICHD) from the 321 survey answers of the BRFSS 2015 data set, using only numpy. The data is very imbalanced (8.8% positives), so we evaluate with the F1 score. Our main finding is that the data preparation, in particular keeping all columns (also the very incomplete ones), helps more than any change of the model. Our final model averages a regularized logistic regression, trained with Newton's method, and three small neural networks. It reaches an F1 score of 0.440 in 5-fold cross validation and 0.454 (accuracy 0.881) on the AIcrowd test set.**

I. Introduction

The BRFSS is a yearly telephone survey of adults in the US about their health and lifestyle. The task is to predict from the answers of a person whether they were ever told that they have coronary heart disease or had a heart attack (MICHD). The training set has $N = 328\,135$ people with 321 features each, the test set has 109 379 people [1].

Only 8.8% of the training labels are positive. A classifier that always says "no" therefore already has an accuracy of 91.2%, which is why the competition uses the F1 score of the positive class. We use it for all our decisions. Only numpy and the standard library are allowed, so all methods are implemented by ourselves.

II. Data Preparation

The columns in BRFSS are survey questions with coded answers, and most of the work was understanding them (all steps are in `preprocessing.py`).

Special codes. The codes for "don't know" and "refused" depend on the width of the field (7/9, 77/99, 777/999) and are replaced by `nan`. The answer "none" (88, 888) becomes 0. A few columns have their own codes, for example 14 for age. Calculated variables (names ending with _) are left alone, because 99 can be a real value there.

Dropped columns. We drop ids, dates, sampling weights, and raw questions where the data set already contains a cleaned version (for example `ALCDAY5`, which mixes days per week and per month).

Encoding. Columns with at most 10 different values (and the state) are one-hot encoded, with `nan` as its own category. The remaining columns are filled with the median, standardized and clipped to $[-5, 5]$ because some have extreme outliers. All statistics come from the training part only. Constant and duplicate columns are removed at the end, which leaves $D = 899$ features and an intercept.

III. Models and Methods

Basic methods. We implemented the six required methods in `implementations.py`: least squares with gradient descent (GD) and stochastic gradient descent (batch size 1), least squares and ridge regression with the normal equations, and logistic and regularized logistic regression with GD. For the least squares methods we predict 1 if $\tilde{x}^\top w > 0.5$.

Newton's method. Plain GD was still far from converged after 1000 steps on our features. For regularized logistic regression with loss $\mathcal{L}(w) = \frac{1}{N}\sum_n \left(\log(1 + e^{x_n^\top w}) - y_n x_n^\top w\right) + \lambda\|w\|^2$ we therefore use Newton's method, $w \leftarrow w - H^{-1}\nabla\mathcal{L}(w)$, which needs only a few iterations (at most 30 in our code). We use $\lambda = 10^{-5}$.

Threshold. The model gives a score $x^\top w$ for each person. Instead of 0 we choose the threshold that maximizes the F1 score on the training scores, which can be done for all thresholds in one sorted pass. This was the most important single step for the classical methods (see Table I), as the default threshold predicts almost no positives.

Neural network. We also implemented a small network in numpy ($D \to 128 \to 64 \to 1$, ReLU, dropout 0.3 [2], cross entropy loss, Adam [3] with learning rate 10^{-3}, weight decay 10^{-4}, batch size 512). We train for 8 epochs and keep the epoch with the best F1 on a validation set.

Blend. The logistic regression and the network are very similar (their scores have a correlation of 0.98), but they do not make exactly the same mistakes. We average the scores of three networks with different seeds, standardize them together with the logistic scores and take the mean. The networks and the logistic regression are trained on 7/8 of the training data, and 1/8 is used for early stopping.

Final threshold. The F1 curve is flat on the validation set (anything between 12% and 18% predicted positives gives the same F1 within noise), so choosing the maximum there is unreliable. Our first blend used it and predicted 15.9% positives. We instead use the positive rate that the logistic regression gets with its threshold tuned on all training samples (13.1%) and set the threshold of the blend to predict the same share on the test set.

Validation. We use 5-fold cross validation with a fixed seed. The threshold is always tuned on the training folds only, never on the fold that is evaluated. Each step below is compared to the same reference, changing only one thing at a time.

Method (5-fold CV)	F1
Required methods, simple preprocessing	
Least squares GD	0.410
Least squares SGD	0.395
Least squares (normal eq.)	0.419
Ridge ($\lambda = 10^{-4}$)	0.419
Logistic GD	0.426
Reg. logistic GD ($\lambda = 10^{-4}$)	0.425
Reg. logistic, Newton ($\lambda = 10^{-5}$)	
Reference	0.428
Class weight 3 / undersampling 1:3	0.428 / 0.427
No code cleaning	0.427
No one-hot encoding	0.425
Neither cleaning nor one-hot	0.416
Top 100 features by correlation	0.423
Polynomial features, degree 2	0.427
Keep all columns	**0.439**
Keep all columns, degree 2	0.440
Hold-out protocol with early stopping	
Logistic regression	0.437
3 networks	0.436
Blend	0.440

Table I
F1 SCORE WITH A THRESHOLD TUNED ON THE TRAINING FOLDS. THE STANDARD DEVIATION OVER THE FOLDS IS 0.002–0.005, SO DIFFERENCES OF 0.002 OR LESS ARE NOT SIGNIFICANT. THE LAST BLOCK USES A DIFFERENT SPLIT, SO IT IS ONLY COMPARABLE WITHIN ITSELF.

IV. RESULTS

Table I shows the cross validation results. With the default threshold the least squares methods have an F1 score of almost zero (0.002 for GD), only the tuned threshold makes them usable. All six required methods are then between 0.395 and 0.426, and the logistic models are better than linear regression.

Newton's method with the logistic loss improves to 0.428. Reweighting the classes or undersampling the negatives does not help once the threshold is tuned, they only move the default threshold. The data preparation is more important: removing the code cleaning and the one-hot encoding together costs 0.012. Neither feature selection nor polynomial features helped.

The largest gain (+0.011) comes from keeping all columns instead of dropping those with many missing values. We did not expect this. Our explanation is that the missing values in survey data are often not random, because many questions are only asked if an earlier answer was positive, so the missingness itself carries information, and the one-hot encoding with a `nan` category lets the model use it. We did not test this explanation further. With all columns, a regularization strength between 10^{-6} and 10^{-4} works equally well, and larger values are worse.

The blend improves the hold-out F1 from 0.437 to 0.440. It is better than the logistic regression in 4 of 5 folds (mean $+0.003$), while the networks alone are not (0.436). This is a small gain, of the same size as the noise level of a single fold.

Submission	positives	F1	Accuracy
Logistic regression	13.1%	0.453	0.881
Blend, threshold from validation set	15.9%	0.448	0.865
Blend, positive rate of logistic	13.1%	**0.454**	0.881

Table II
SCORES ON THE AICROWD TEST SET.

On the AIcrowd test set (Table II) the first blend with its validation threshold was worse than the logistic regression (0.448), because it predicts too many positives. With the threshold from the training rate, the blend reaches 0.454 and an accuracy of 0.881. The difference to the logistic regression (0.453) is not significant. We submitted the last version and it is what `run.py` produces.

V. DISCUSSION

Our estimates of the test F1 (0.440 in cross validation) are lower than the leaderboard (0.454), which we cannot explain except by the difference between the splits. Since F1 is sensitive to the threshold and the F1 curve is flat, it is easy to overfit the threshold, and we found that a rate based on much more data was more reliable than the argmax on a small validation set.

A limitation is that we chose the positive rate for the blend after seeing the first leaderboard scores, so the leaderboard number is slightly optimistic. The other conclusion we are confident about is that data preparation mattered more than the model. A natural next step would be a different kind of model (for example trees), as the network and the logistic regression make almost the same predictions.

VI. SUMMARY

For predicting heart disease from BRFSS, the biggest improvements came from cleaning the answer codes, one-hot encoding with a missing category, keeping all columns and tuning the decision threshold for F1. A blend of a logistic regression and small networks gave a small further gain. The code is at https://github.com/lb2002/projects (folder `project_1`). Running `python run.py v4_blend` in this folder with the data in `data/` writes the submitted file (about 12 minutes), and `cross_validation.py` produces Table I.

REFERENCES

[1] Centers for Disease Control and Prevention, "Behavioral risk factor surveillance system, 2015 annual data," 2015, https://www.cdc.gov/brfss/annual_data/annual_2015.html.

[2] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.